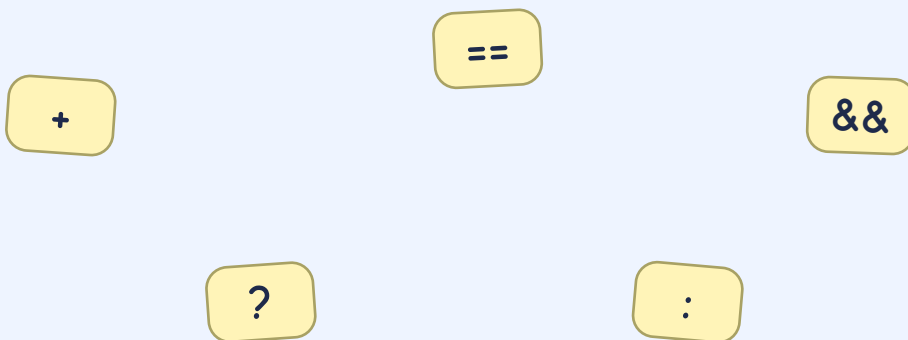


JavaScript

Session 2

Operators, Expressions & Logic



read -> reason -> predict -> run -> explain

Rewritten in simple language, with examples, practice and revision

02 Notebook Index

click any line to jump

03	Session roadmap	22	Logical operators
04	Operator, operand and expression	23	Logical truth tables
05	Arithmetic operators	24	Short-circuit evaluation
06	Arithmetic in action	25	Truthy, falsy and returned operands
07	Remainder (%)	26	Ternary operator
08	Operator precedence and parentheses	27	Ternary vs if/else
09	Increment and decrement overview	28	Problem solving: sign check
10	Pre-increment (++x)	29	Problem solving: largest number
11	Post-increment (x++)	30	Problem solving: even or odd
12	Pre vs post comparison	31	Problem solving: access check
13	Execution tracing	32	Common mistakes
14	Compound ++ expression	33	Practice set A
15	Decrement (--x / x--)	34	Practice set B
16	Assignment operators	35	Answer key A
17	Assignment practice	36	Answer key B
18	Comparison operators	37	Interview revision
19	Loose vs strict equality	38	One-page quick revision

HOW TO USE THIS NOTEBOOK

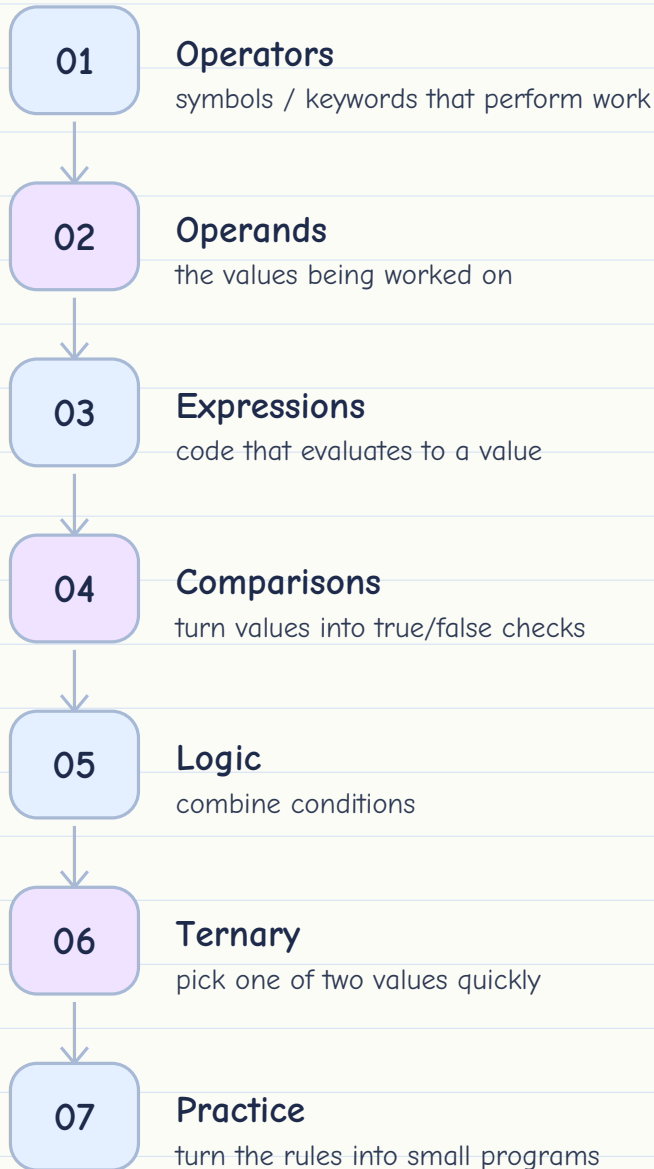
This index is the fast route. For study, go page by page the first time. Later, use the quick revision page as your last-minute sheet.

03 Session Roadmap

from symbols to problem solving

BIG PICTURE

Session 1 taught us how to store values. Session 2 is about what we can DO with those values: calculate, compare, combine conditions and choose a result.



MY RULE FOR THIS SESSION

Study trick: every time an operator appears, ask two questions: "What value comes in?" and "What value comes out?" That habit makes ++, == and && much easier to trace.

04 Operator, Operand, Expression

three words to keep separate

TINY EXAMPLE

`10 + 5`

Operator

The symbol that tells JavaScript what to do.

+ means add.

Operands

The values that the operator works on.

10 and 5 are the operands.

Expression

A piece of code that evaluates to a value.

`10 + 5` becomes 15.

REMEMBER

An expression can be as small as `5 + 2`, or bigger, like `age >= 18 ? "adult" : "minor"`. The important part is that JavaScript can evaluate it and obtain a result.

05 Arithmetic Operators

the operators you use for calculations

Operator	Name	What it does	Example
+	addition	adds values	8 + 3 -> 11
-	subtraction	subtracts	8 - 3 -> 5
*	multiplication	multiplies	8 * 3 -> 24
/	division	divides	8 / 4 -> 2
%	remainder	gives the remainder	8 % 3 -> 2
**	power	raises a number to a power	2 ** 3 -> 8
++	increment	adds 1	x++
--	decrement	subtracts 1	x--

THREE EASY TRAPS

`/` can produce a decimal. Example: `5 / 2` gives `2.5`. `\*\*` is power, so `3 \*\* 2` means 3 raised to 2 = 9. `%` is not a percentage operator - it is remainder.

06 Arithmetic in Action

write the result into a variable, then print it

BEST BEGINNER EXAMPLE

```
let price = 120;  
let quantity = 3;  
let total = price * quantity;  
  
console.log("Total:", total);
```

HOW TO READ THE CODE

Step 1 -> `price` stores 120. Step 2 -> `quantity` stores 3. Step 3 -> `price \* quantity` calculates 360. Step 4 -> `total` stores 360. Step 5 -> `console.log()` prints it.

QUICK PRACTICE

```
let a = 2;  
let b = 4;  
console.log(a + b);  
console.log(a - b);  
console.log(a * b);  
console.log(a / b);  
console.log(a ** b);
```

CHECK YOURSELF

Expected values: 6, -2, 8, 0.5 and 16. Do not memorize the numbers - learn what each symbol asks JavaScript to calculate.

07 Remainder Operator (%)

the quickest path to divisibility checks

START HERE

```
console.log(10 % 3);  
// 1
```

10 / 3 quotient = 3 remainder = 1

WHY % IS USEFUL

When checking even/odd, divide by 2. If the remainder is 0, the number is even. If the remainder is 1 (for whole numbers), it is odd.

EVEN / ODD CHECK

```
let num = 42;  
const result = num % 2 === 0;  
console.log(result); // true  
  
let other = 21;  
console.log(other % 2 === 0); // false
```

PATTERN TO REMEMBER

The pattern is: `value % divisor === 0`. If the comparison is true, the value is exactly divisible by that divisor.

08 Operator Precedence

when more than one operator appears

FIRST EXAMPLE

```
let result = 2 + 3 * 4;  
console.log(result); // 14
```

WHAT HAPPENED?

JavaScript does not simply read every operator from left to right. Some operators are applied before others. Multiplication happens before addition here, so `3 \* 4` becomes 12, then 2 + 12 becomes 14.

USE PARENTHESES WHEN YOU WANT A DIFFERENT GROUPING

```
let result = (2 + 3) * 4;  
console.log(result); // 20
```

NOTEBOOK RULE

Parentheses are your clarity tool. Even when JavaScript already knows the precedence, parentheses can make your intention obvious to another programmer.

Small beginner priority list

- parentheses `(` `)`
- power `\*\*`
- multiplication / division / remainder `\*` `/` `%`
- addition / subtraction `+` `-`
- comparisons such as `>`, `<`, `===`
- logical `&&`, then `||`
- ternary `?` `:`

09 Increment and Decrement

same symbol, different timing

CORE IDEA

`++` adds 1. `--` subtracts 1. The important question is not only "what changes?" but also "when is the new value used?"

Form	What changes?	What value does the expression produce?
++x	x increases first	new value
x++	x increases after	old value
--x	x decreases first	new value
x--	x decreases after	old value

NORMAL USE: JUST CHANGE THE VARIABLE

```
let score = 10;  
score++;  
console.log(score);  
// 11
```

GOOD HABIT

Most beginner code is easier to read when you use `score++` or `score += 1` on its own line. The pre/post difference matters when the expression itself is used to produce a value.

10 Pre-Increment (++x)

change first, use second

TRACE THIS SLOWLY

```
let x = 2;  
console.log(++x);  
console.log(x);
```

Start: x = 2

1. ++x -> x becomes 3
2. expression value -> 3
3. x stays 3 after the statement

ONE-LINE MEMORY

Think of `++x` as: "increase x, then hand me the updated value."

STORED RESULT EXAMPLE

```
let points = 9;  
let shown = ++points;  
console.log(shown); // 10  
console.log(points); // 10
```

11 Post-Increment (x++)

use the old value, then change the variable

TRACE THIS SLOWLY

```
let x = 4;  
console.log(x++);  
console.log(x);
```

Start: x = 4

1. expression value -> 4
2. then x becomes 5
3. next console.log(x) prints 5

ONE-LINE MEMORY

Think of `x++` as: "give me the current x, then increase x."

STORED RESULT EXAMPLE

```
let count = 7;  
const before = count++;  
console.log(before); // 7  
console.log(count);  // 8
```

12 Pre vs Post - Side by Side

same update, different returned value

Code	Step 1	Step 2	Value produced
++x	x = x + 1	use x	new x
x++	use x	x = x + 1	old x
--x	x = x - 1	use x	new x
x--	use x	x = x - 1	old x

COMPARE THE OUTPUTS

```
let n = 5;
console.log(++n); // 6
console.log(n);   // 6
```

```
let m = 5;
console.log(m++); // 5
console.log(m);   // 6
```

IMPORTANT

If `++x` or `x++` is on its own line, both increase x by 1. The difference only becomes visible when JavaScript needs the value produced by the expression.

13 Execution Tracing

make a tiny table instead of guessing

EXAMPLE TO TRACE

```
let x = 3;  
console.log(++x, x++);  
console.log(x);
```

Step	Expression	x before	value produced	x after
1	++x	3	4	4
2	x++	4	4	5

RESULT

The output is `4 4` on the first line and `5` on the second. When a pre/post operator confuses you, write the variable state before and after.

Tracing recipe

- write the starting value
- read one operator from left to right
- write the value produced
- update the variable
- move to the next operator

14 Compound ++ Expression

slow down - one operator at a time

SOURCE-INSPIRED CHALLENGE

```
let x = 5;
let y = 5;

const result = x++ + y++ + ++y + y++;

console.log(result);
console.log(y);
```

Part	value used	variable after
x++	5	x = 6
y++	5	y = 6
++y	7	y = 7
y++	7	y = 8

HOW TO SOLVE IT

Add the values that were USED by each expression: $5 + 5 + 7 + 7 = 24$. The final y is 8. Do not try to calculate the whole thing in one jump.

READABLE-CODE NOTE

This kind of dense expression is useful for learning how the operator works, but clear production code is usually easier to maintain when the updates are split into separate lines.

15 Decrement Operators

the same timing rule, but subtract 1

PRE AND POST DECREMENT TOGETHER

```
let value = 5;  
console.log(--value); // 4  
console.log(value--); // 4  
console.log(value);   // 3
```

READ THE ORDER

`--value` changes first, so the expression gives 4. `value--` gives the current 4 first, then changes the variable to 3.

Form	Meaning
--x	$x = x - 1$, then use x
x--	use x , then $x = x - 1$

MIRROR RULE

Increment and decrement are mirror images: `++` adds one, `--` subtracts one. The pre/post timing rule is identical.

16 Assignment Operators

calculate and save the new value

Operator	Same as	Simple meaning
=	$x = y$	put a value into x
+=	$x = x + y$	add, then store
-=	$x = x - y$	subtract, then store
*=	$x = x * y$	multiply, then store
/=	$x = x / y$	divide, then store
%=	$x = x \% y$	remainder, then store
**=	$x = x ** y$	power, then store

SHORT FORM VS LONG FORM

```
let total = 30;
let extra = 40;

total += extra;
console.log(total); // 70

// same idea written longhand
total = total + extra;
```

REMEMBER

The shortcut is not a brand-new operation. It is a compact way of saying: calculate with the current value and assign the result back to the same variable.

17 Assignment Practice

watch the variable change after every line

WALK THROUGH THE STATE

```
let balance = 30;  
let change = 40;  
  
balance += change;  
balance -= change;  
balance *= change;  
balance /= change;
```

Step	Operation	balance after
start	-	30
1	+= 40	70
2	-= 40	30
3	*= 40	1200
4	/= 40	30

READING METHOD

Shortcut thinking: first look at the left variable, then look at the right value, then perform the operation. Finally store the result back in the left variable.

18 Comparison Operators

comparisons usually answer with true or false

Operator	Meaning	Example
==	loose equality	5 == "5" -> true
===	strict equality	5 === "5" -> false
!=	loose inequality	5 != "5" -> false
!==	strict inequality	5 !== "5" -> true
>	greater than	10 > 3 -> true
<	less than	10 < 3 -> false
>=	greater or equal	10 >= 10 -> true
<=	less or equal	10 <= 10 -> true

WHY COMPARISONS MATTER

Comparison operators are the bridge to conditions. Example: `marks >= 40` is a question. JavaScript evaluates it and gets either true or false.

SMALL CONDITION

```
let marks = 72;  
const passed = marks >= 40;  
console.log(passed); // true
```

Loose vs Strict Equality

the famous == vs === question

SAME VISIBLE CHARACTERS, DIFFERENT TYPES

```
console.log(22 == "22");  
console.log(22 === "22");
```

Expression	Result	Why?
22 == "22"	true	loose comparison may convert types
22 === "22"	false	strict comparison keeps the type difference

MENTAL PICTURE

A number `22` and a string `"22"` are not the same type. Strict equality asks for both a matching value and a matching type.

WHY STRICT EQUALITY IS EASIER TO REASON ABOUT

```
let userInput = "18";  
let age = 18;  
  
console.log(userInput == age); // true  
console.log(userInput === age); // false
```

STUDY NOTE

For beginner code, using `===` makes the type check visible instead of hiding a conversion inside the comparison.

20 Inequality Operators

the opposite side of equality

SAME PAIR, NOW CHECKING "DIFFERENT"

```
console.log(10 != "10");  
console.log(10 !== "10");
```

Operator	Think of it as
!=	not equal after loose comparison
!==	not equal under strict comparison

EXAMPLE EXPLAINED

`10 != "10"` is false because loose equality can consider them equal after conversion.
`10 !== "10"` is true because the types differ.

QUICK TEST

```
const entered = "10";  
const expected = 10;  
  
console.log(entered !== expected); // true
```

21 Relational Operators

greater, smaller, or equal in one direction

SAME PAIR AS THE LESSON

```
let a = 22;  
let b = "22";  
  
console.log(a > b);  
console.log(a < b);  
console.log(a >= b);  
console.log(a <= b);
```

Expression	Result	Plain-language reason
22 > "22"	false	22 is not greater than 22
22 < "22"	false	22 is not smaller than 22
22 >= "22"	true	they compare as equal in value
22 <= "22"	true	they compare as equal in value

IMPORTANT

When different types are compared with `<`, `>`, `<=` or `>=`, JavaScript may convert values before comparing them. Learn the result AND the rule that produced it.

22 Logical Operators

combine conditions into one decision

Operator	Name	Beginner rule
&&	AND	both sides must be true
	OR	at least one side must be true
!	NOT	flip the Boolean result

REAL-WORLD ACCESS EXAMPLE

```
let hasPassword = true;
let isVerified = false;

console.log(hasPassword && isVerified); // false
console.log(hasPassword || isVerified); // true
console.log(!isVerified); // true
```

SIMPLE PICTURE

AND is like a gate with two locks: both conditions must open it. OR is like two different entry routes: one working route is enough. NOT changes true to false, and false to true.

23 Logical Truth Tables

the fastest way to memorize boolean combinations

AND

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

OR

A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

NOT

A	!A
true	false
false	true

ONE SENTENCE

Memory line: AND needs all true. OR needs at least one true. NOT simply flips the Boolean value.

24 Short-Circuit Evaluation

sometimes JavaScript stops early

WHAT "SHORT-CIRCUIT" MEANS

Logical operators are evaluated from left to right. If the answer is already known, JavaScript may not evaluate the remaining side.

WATCH THE RIGHT-HAND SIDE

```
false && console.log("this never runs");  
true || console.log("this never runs");
```

TWO MEMORY PATTERNS

`false && anything` is already false, so there is no reason to evaluate `anything`. `true || anything` is already true, so JavaScript can stop there too.

PROOF BY SIDE EFFECT

```
function check(){  
  console.log("check ran");  
  return true;  
}  
  
console.log(true || check());
```

WHY YOU SHOULD CARE

The function call on the right is skipped because the left side of `||` is already truthy. This is useful for understanding both performance and side effects.

25 Truthy, Falsy and Returned Values

a small but important upgrade to the basic AND/OR rule

THE DEEPER RULE

In simple Boolean examples, `&&` and `||` look like they return true or false. In real JavaScript, they can return one of their actual operands. What decides the path is truthiness.

Value	Falsy?
false	yes
0	yes
""	yes
null	yes
undefined	yes
NaN	yes
42	no
"hello"	no
[]	no
{}	no

LOGICAL OPERATORS CAN SELECT VALUES

```
console.log("" || "Guest");  
// "Guest"
```

```
console.log(5 && 8);  
// 8
```

```
console.log(0 && 8);  
// 0
```

EXAM-SAFE EXPLANATION

For the session-level rule, learn the Boolean cases first. Then remember this advanced fact: `&&`, `||` and `??` are value-selection operators when used with non-Boolean operands.

26 Ternary Operator

a compact way to choose between two values

BEST BEGINNER EXAMPLE

```
const age = 21;  
const status = age >= 18 ? "Adult" : "Minor";  
console.log(status);
```

condition ? if true : if false

HOW TO READ IT

Read it as: "If the condition is true, use the value after `?`. Otherwise, use the value after `:`." The whole ternary expression produces one value, which can be stored in a variable.

ANOTHER PRACTICAL EXAMPLE

```
const marks = 76;  
const result = marks >= 40 ? "Pass" : "Fail";  
console.log(result); // Pass
```

27 Ternary vs if / else

same decision, different shape

TERNARY

```
const result = age >= 18 ? "Adult" : "Minor";
```

IF / ELSE

```
let result;  
if (age >= 18) {  
  result = "Adult";  
} else {  
  result = "Minor";  
}
```

READABLE-CODE RULE

Use a ternary when the decision is short and you simply need one of two values.

Use a normal `if / else` block when the logic has multiple steps or becomes harder to read.

DO NOT OVERUSE IT

Avoid turning one line into a puzzle by nesting many ternaries inside each other. Shorter code is not automatically clearer code.

28 Problem Solving: Positive or Negative

comparison + ternary

SOLUTION

```
const number = -10;  
const sign = number >= 0 ? "Positive" : "Negative";  
console.log(sign);
```

EXPLAIN THE CODE

`number >= 0` is false for -10, so JavaScript chooses the value after `:`. The final value is `"Negative"`.

SLIGHTLY BETTER REUSABLE VERSION

```
function getSign(number){  
    return number >= 0 ? "Positive" : "Negative";  
}  
  
console.log(getSign(25)); // Positive  
console.log(getSign(-3)); // Negative
```

PATTERN

The important problem-solving pattern is more general than this example: build a Boolean question first, then decide what text or value should be used when the answer is true or false.

Problem Solving: Largest Number

comparison + ternary, one decision

BETTER OUTPUT: RETURN THE ACTUAL NUMBER

```
const a = 10;  
const b = 20;  
  
const larger = a > b ? a : b;  
console.log(larger); // 20
```

READ THE LOGIC

The condition `a > b` is false because 10 is not greater than 20. So the ternary gives us `b`, which is 20.

REALISTIC EXAMPLE

```
const first = 45;  
const second = 32;  
const largest = first > second ? first : second;  
  
console.log("Largest:", largest);
```

WHY THIS VERSION IS USEFUL

A useful improvement over the source-style example is to return the number itself rather than only printing which variable is larger. That makes the result reusable later.

30 Problem Solving: Even or Odd

modulus + strict equality + ternary

CLEAN SOLUTION

```
const number = 21;  
const type = number % 2 === 0 ? "Even" : "Odd";  
console.log(type);
```

21 % 2 **-> 1** **-> 1 === 0 is false** **-> Odd**

THREE OPERATORS WORKING TOGETHER

The `%` operator finds the remainder. The `=== 0` part turns that remainder into a Boolean check. The ternary turns the Boolean check into a readable result.

TINY PRACTICE LOOP

```
for (const n of [12, 17, 40, 55]) {  
  const kind = n % 2 === 0 ? "Even" : "Odd";  
  console.log(n, kind);  
}
```

Problem Solving: Access Check

logical AND in a real rule

TWO LOCKS, ONE GATE

```
const hasPassword = true;
const isVerified = true;

const canEnter = hasPassword && isVerified;
console.log(canEnter); // true
```

EXPLAIN IT

Because both conditions are true, AND produces true. If either condition becomes false, the final Boolean result becomes false.

OR EXAMPLE

```
const isAdmin = false;
const isOwner = true;

const canManage = isAdmin || isOwner;
console.log(canManage); // true
```

PROBLEM-SOLVING PATTERN

This is how logical operators connect simple checks into a larger rule: one condition describes one fact; `&&` or `||` describes how those facts must work together.

32 Common Mistakes

the things that usually cause wrong answers

1. Confusing ``=`` with ``===``
``=`` assigns. ``===`` compares.

2. Forgetting that ``x++`` produces the old value
The variable changes after the expression produces its value.

3. Reading ``%`` as "percent"
In JavaScript it is the remainder operator.

4. Treating ``&&`` and ``||`` as always returning Boolean
With non-Boolean values, they can return an operand.

5. Writing a long ternary puzzle
If it becomes hard to read, use ``if / else``.

6. Doing mental math on a compound `++` expression
Trace one operator at a time.

7. Ignoring parentheses
Use them when grouping should be obvious.

Practice Set A

try without looking at the answer key

1. Write an expression that adds 18 and 7.
2. What is the result of `17 % 5`?
3. Predict the output: `let x = 4; console.log(x++); console.log(x);`
4. Convert `score += 10` into its longer form.
5. What is the difference between `==` and `===`?
6. Predict: `10 !== "10"`.
7. Predict: `12 >= "12"`.
8. Write a check that returns true when a number is even.
9. Write an AND condition for `hasPassword` and `isVerified`.
10. Write a ternary that labels age 18 or more as `Adult`.

34 Practice Set B

slightly more thinking, still the same operators

11. Predict: ``true && false || true``.

12. Predict: ``false || "Guest"``.

13. Explain why ``2 + 3 * 4`` is 14, not 20.

14. Rewrite ``total = total * 2`` using a compound assignment operator.

15. Write code that returns the smaller of two numbers using a ternary.

16. Write code that prints ``Pass`` when marks are 40 or more, otherwise ``Fail``.

17. Show the steps for ``let x = 5; console.log(++x, x++);``.

18. Give one example where ``||`` can provide a fallback value.

19. Why can short-circuiting matter when the right side has a function call?

20. Write one small program that uses `%`, `===`, `&&` and `?:` together.

Answer Key A

solutions with the reason, not just the output

1. `18 + 7`

The expression evaluates to 25.

2. `17 % 5`

2, because 15 goes into the division and 2 remains.

3. `5` then `5`?

For ``x = 4``, `x++` prints 4 and then `x` becomes 5. The second print is 5.

4. `score = score + 10`

That is the long form of ``score += 10``.

5. `==` vs `===`

``==`` may coerce types; ``===`` compares without converting the operands.

6. `true`

10 and "10" differ in type, so strict inequality is true.

7. `true`

The relational comparison can coerce the string "12" to a numeric value for comparison.

8. `number % 2 === 0`

A zero remainder means the number is divisible by 2.

9. `hasPassword && isVerified`

Both facts must be true.

10. `age >= 18 ? "Adult" : "Minor"`

The condition selects the correct text value.

Answer Key B

the useful reasoning behind the harder set

11. true

'&&' is evaluated before '||': true && false -> false; false || true -> true.

12. "Guest"

The empty string is falsy, so OR moves to the right-hand value.

13. 14

Multiplication happens before addition: $3 * 4 = 12$, then $2 + 12 = 14$.

14. total *= 2

That is the compound-assignment form.

15. $a < b ? a : b$

If a is smaller, use a; otherwise use b.

16. marks $\geq 40 ?$ "Pass" : "Fail"

The comparison creates the decision.

17. 6, 6

Starting at 5: ++x makes 6 and produces 6; x++ then produces 6 and moves x to 7 after the expression.

18. name || "Guest"

When name is falsy, the fallback string is selected.

19. The call may be skipped

If the left side already determines the result, the right side is not evaluated.

20. Example below

Use all four pieces in one small program.

37 Interview Revision

short answers to say confidently

What is an operator?

A symbol or keyword that performs an operation on one or more operands.

What is an operand?

A value that an operator works with.

What is an expression?

A piece of code that JavaScript evaluates to produce a value.

What does % do?

It returns the remainder after division.

`++x` vs `x++`?

`++x` increments first and produces the new value. `x++` produces the old value first, then increments.

`+=` vs `+`?

`+` calculates a sum. `+=` calculates a sum and stores the new value back in the left variable.

`==` vs `===`?

Loose equality may coerce types. Strict equality compares without type conversion.

`&&` vs `||`?

AND needs all conditions to be true; OR needs at least one. Both can short-circuit.

What is short-circuiting?

Stopping the evaluation when the final logical result is already known.

What is a ternary?

A three-part expression: `condition ? trueValue : falseValue`.

38 One-Page Quick Revision

last-minute scan before a quiz or viva

ARITHMETIC

+ - * / % **
++ --

ASSIGNMENT

= += -= *= /= %= **=

COMPARISON

== === != !==
> < >= <=

LOGICAL

&& || !
AND / OR / NOT

TERNARY

condition ?
valueIfTrue : valueIfFalse

TRACE

pre -> change first
post -> use first

TEN-SECOND SCAN

Fast memory: `=` assigns. `===` compares. `%` finds remainder. `++x` changes first.
`x++` uses first. `&&` needs all true. `||` needs one true. `!` flips. `?:` chooses one value.

EXAM HABIT

Before answering an output question, write the variable state. Before choosing an operator, say what you want the program to do in plain English.

Clean Session 2 Example

a rewritten practice program, not the source file

CLEAN COMBINED PRACTICE

```
console.log("=== Operators Practice ===");

const price = 120;
const quantity = 3;
const total = price * quantity;
console.log("Total:", total);

let score = 10;
console.log("Before:", score);
console.log("Post:", score++);
console.log("After:", score);
console.log("Pre:", ++score);

let points = 30;
points += 20;
points -= 5;
console.log("Points:", points);

const typedAge = "18";
console.log(typedAge == 18);
console.log(typedAge === 18);

const hasPassword = true;
const isVerified = true;
const canEnter = hasPassword && isVerified;
console.log("Access:", canEnter);

const number = 21;
const kind = number % 2 === 0 ? "Even" : "Odd";
console.log("Kind:", kind);

const a = 14;
const b = 9;
const larger = a > b ? a : b;
console.log("Larger:", larger);
```

WHAT THIS PROGRAM DEMONSTRATES

This final example combines the session ideas into one readable file: arithmetic, pre/post operators, compound assignment, strict/loose equality, AND, modulus and ternary. The variables are named by meaning instead of single letters, so the program is easier to read later.

40 Reference Shelf + Final Checklist

the notebook is complete

HOW THIS NOTEBOOK WAS BUILT

The three uploaded files were used as the structure and topic reference. The notes were rewritten in fresh wording and examples, while the source sequence of arithmetic, increment/decrement, assignment, comparison, logic and ternary was preserved.

Web reference shelf

[MDN - Expressions and operators](#)

[MDN - Logical AND](#)

[MDN - Logical OR](#)

[MDN - Ternary operator](#)

[W3Schools - JavaScript operators](#)

[W3Schools - Operators reference](#)

FINAL SELF-CHECK

1. Can I explain what each operator does? 2. Can I predict pre/post output without running the code? 3. Can I explain `==` vs `===` in one sentence? 4. Can I write an AND / OR condition? 5. Can I solve even/odd using `%`? 6. Can I write a small ternary? 7. Can I explain why short-circuiting happens? 8. Can I trace a variable line by line?

FINISH LINE

You are ready to move on when you can explain the output, not just memorize it. The next JavaScript topic will build on these operators as building blocks for conditions, loops and functions.

Reference websites checked for operator behavior and examples: MDN + W3Schools